Internship Project Documentation Report

SamiX

GenAI-Powered Customer Support Quality Auditor

Submitted By:

Anand Kumar

E V Udayasree

Varshasree

Guru Krishna

Under the Guidance of

Mr. Jonathan Vineeth

INDEX

1. Executive Summary	
2. Project Overview	
2.1 Problem Statement	
2.2 Core Capabilities	
2.3 Technology Stack	
3. System Architecture	
3.1 High-Level Architecture	
3.2 Audit Pipeline (Step-by-Step)	
3.3 Authentication State Machine	
4. Project Structure	
5. Setup and Installation	
5.1 Prerequisites	
5.2 Quick Setup (Windows)	
5.3 API Key Configuration	
5.4 Milvus Lite Setup	
6. Feature Walkthrough	
6.1 Authentication Flow	
6.2 New Audit (Upload Page)	
6.3 Session Detail (Results)	
6.4 QA Dashboard	
6.5 Admin Panel	
7. **Module Reference**	
7.1 Groq Client (LLM Inference)	
7.2 Speech-to-Text Processor	
7.3 Knowledge Base Manager (RAG)	

7.4 History Manager	
7.5 UI Components	
API Keys and Configuration	
8.1 Required and Optional Keys	
8.2 Environment Variables	
8.3 Streamlit Secrets	
9. **Deployment**	
9.1 Local Development	
9.2 Streamlit Cloud	
9.3 Docker Deployment	
9.4 Environment Matrix	
10. Troubleshooting	
Data Schemas	
11.1 Audit Result Schema	
11.2 User Record Schema	
Team Contributions	
Limitations	
Scalability Considerations	
12. Future Enhancements	
Conclusion	
Repository Links	

INTRODUCTION

SamiX is a GenAI-powered system designed to automate the quality auditing of customer support interactions. In traditional call center environments, less than 5% of customer conversations are manually reviewed, leading to inconsistent evaluations, undetected policy violations, and delayed feedback. SamiX addresses these challenges by leveraging advanced technologies such as Speech-to-Text (Deepgram/Whisper), Retrieval-Augmented Generation (RAG), and Large Language Models (Groq Llama-3) to analyze 100% of customer interactions efficiently.

The system transcribes audio calls, identifies speakers, retrieves relevant organizational policies, and evaluates agent performance through a dual-pass scoring mechanism. It also detects critical errors, provides corrective suggestions, and generates actionable insights via an interactive dashboard. By delivering audit results in under 30 seconds, SamiX ensures real-time feedback, improved compliance, and enhanced customer satisfaction. The modular and scalable architecture makes it suitable for enterprise deployment, enabling organizations to optimize support quality while reducing operational costs.

Additionally, SamiX provides administrative capabilities such as knowledge base management, user monitoring, and cost tracking for API usage, offering complete visibility into system performance and operational efficiency. Its extensible design allows integration with CRM platforms and supports future enhancements such as real-time call monitoring and multi-language support. This makes SamiX not only a technical solution but also a strategic tool for improving decision-making and driving continuous improvement in customer service operations.

1. Project Overview

SamiX (Samiksha — 'The All-Seeing Eye of Quality') is a GenAI-powered customer support quality auditor. It automatically transcribes customer support calls, scores agent performance against company policies using Retrieval-Augmented Generation (RAG), and surfaces actionable insights through a professional web dashboard.

1.1 Problem Statement

In large-scale customer support operations, quality assurance (QA) is primarily manual, time-consuming, and limited in coverage. Typically, less than 5% of customer interactions are audited due to resource constraints.

This leads to several challenges:

- Inconsistent evaluation of agent performance
- Undetected policy violations
- Delayed feedback for improvement
- Poor customer experience (CSAT)

As a result, organizations fail to identify systemic issues and lose opportunities to improve service quality. There is a need for an automated, scalable, and consistent solution to evaluate 100% of customer interactions efficiently.

1.2 Core Capabilities

- Automatic speech-to-text transcription with speaker diarization (Agent vs. Customer)
- Policy-aware dual-pass QA scoring using Groq Llama-3 and company-specific RAG knowledge base
- Wrong-turn detection: exact quote, what went wrong, and the correct response
- Real-time analytics dashboard with score trends, radar charts, and history
- Live call mode with real-time RAG suggestions to the agent during the call
- Admin panel for knowledge base management, user oversight, and billing

1.3 Technology Stack

Layer	Technology	Role
Frontend	Streamlit (Python)	Web UI — dark-themed SaaS dashboard
LLM Inference	Groq (Llama-3.3-70B)	Dual-pass QA scoring + summarisation
Speech-to-Text	Deepgram (cloud)	STT with speaker diarization
STT Fallback	OpenAI Whisper (local)	Offline / backup transcription
Vector DB	Milvus Lite	RAG knowledge base — policy storage
Embeddings	HuggingFace all-MiniLM-L6-v2	Sentence embeddings for RAG search
RAG Framework	LangChain	Document chunking, retrieval chain
Audio Processing	pydub + FFmpeg	WAV 16 kHz conversion
Text-to-Speech	gTTS / pyttsx3	Audio summary synthesis
Auth	streamlit-authenticator + bcrypt	Session management + password hashing
Deployment	Docker / Streamlit Cloud	Container-first cloud deployment

2. System Architecture

SamiX is structured as a layered Python application. The Streamlit frontend communicates with a set of service modules — pipeline, utils, auth — each responsible for a single concern. All layers are decoupled through well-defined interfaces, making individual components replaceable without affecting the rest of the system.

2.1 High-Level Architecture

Architecture layers (top to bottom)

Layer 1 — Presentation: Streamlit UI (app.py + src/ui/)
Layer 2 — Auth: AuthManager (src/auth/authenticator.py)
Layer 3 — Pipeline: GroqClient · STTProcessor · AlertEngine (src/pipeline/)
Layer 4 — Utilities: KBManager · HistoryManager · CostTracker · AudioProcessor (src/utils/)
Layer 5 — Data: data/ — JSON history · YAML users · Milvus vectors · uploaded files
Layer 6 — External APIs: Groq API · Deepgram API

2.2 Audit Pipeline — Step by Step

When a user uploads an audio file or pastes a transcript, the following pipeline executes sequentially:

Step	Module	What Happens
1. Ingest	AudioProcessor	pydub converts uploaded file to WAV 16 kHz mono — the format required by Deepgram and Whisper
2. Transcribe	STTProcessor	Deepgram cloud STT transcribes the audio with diarize=true — separating Agent and Customer turns. Falls back to local Whisper if Deepgram is unavailable
3. RAG search	KBManager	The transcript summary is embedded via all-MiniLM-L6-v2 and used to retrieve the top-k relevant policy chunks from the Milvus Lite vector store
4. Score (pass 1)	GroqClient	Groq Llama-3 receives the transcript + retrieved policy context and generates: QA score 0–100, empathy score, compliance score, summary, issues list, recommendations
5. Score (pass 2)	GroqClient	A second Groq call identifies exact wrong turns — verbatim quote, what policy was violated, and the correct response the agent should have given
6. Store	HistoryManager	The full result JSON is saved to data/history/ with the original filename preserved throughout

Step	Module	What Happens
7. Alert	AlertEngine	If QA score < 60, st.toast fires a screen notification and optionally sends an SMTP email alert
8. Cost track	CostTracker	Token counts are recorded; cost = tokens × per-token price. Profit/loss margin is calculated against the configured billing rate

2.3 Auth State Machine

The application uses a 4-state authentication router. The state is stored in st.session_state and evaluated on every render:

State	Screen Shown
landing	Marketing landing page — hero, features, CTAs (unauthenticated root)
login	Sign in form — split panel with social proof on the left
signup	Create account form — split panel with feature highlights on the left
app	Full dashboard — sidebar navigation, page router, all panels

4. Setup and Installation

4.1 System Requirements

Component	Requirement
Operating System	Windows / Linux / macOS
Python	3.11 or higher
RAM	Minimum 8 GB (16 GB recommended)
Storage	At least 2 GB free space
Internet	Required for API-based services

4.2 Prerequisites

Requirement	Version	Notes
Python	3.11+	Required — match runtime.txt
FFmpeg	Any	Required for pydub audio conversion — must be in system PATH
Git	Any	For cloning the repository
Groq API key	Free tier	~14,400 requests/day — get at console.groq.com
Deepgram API key	Free tier ($200 credit)	Optional — Whisper fallback available

4.2 Quick Setup (Windows)

```
git clone https://github.com/evudayasree03/Customer-Quality-Support.git
cd Customer-Quality-Support
python -m venv venv    #For Linus/MacOS: python3 -m venv venv
venv\Scripts\activate  #For Linux/MacOS: source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python generate_hash.py
streamlit run app.py
```

4.3 API Key Configuration

Create a .env file in the project root (this file is git-ignored):

```
GROQ_API_KEY=gsk_your_key_here
DEEPGRAM_API_KEY=your_deepgram_key_here   # optional
```

Or add to .streamlit/secrets.toml for Streamlit Cloud deployment:

```
[api_keys]
groq_api_key = "gsk_your_key_here"
deepgram_api_key = "your_key_here"
```

Important security notes

Never commit .env or secrets.toml to git — both are listed in .gitignore.
The secrets.toml file also stores bcrypt-hashed user passwords.
Run python generate_hash.py to generate a new bcrypt hash for any password.
The Groq API key must start with 'gsk_' — mock data is shown if the key is missing or invalid.

4.5 Milvus Lite (Vector DB)

Milvus Lite is the recommended vector store. On Windows, install the native binaries:

```
pip install "pymilvus[milvus_lite]"
```

If Milvus Lite fails on Windows (native binary issue), the system automatically falls back to keyword search. The app remains fully functional — only semantic similarity search is degraded.

NOTE: If installation fails, the system will fallback to keyword search.

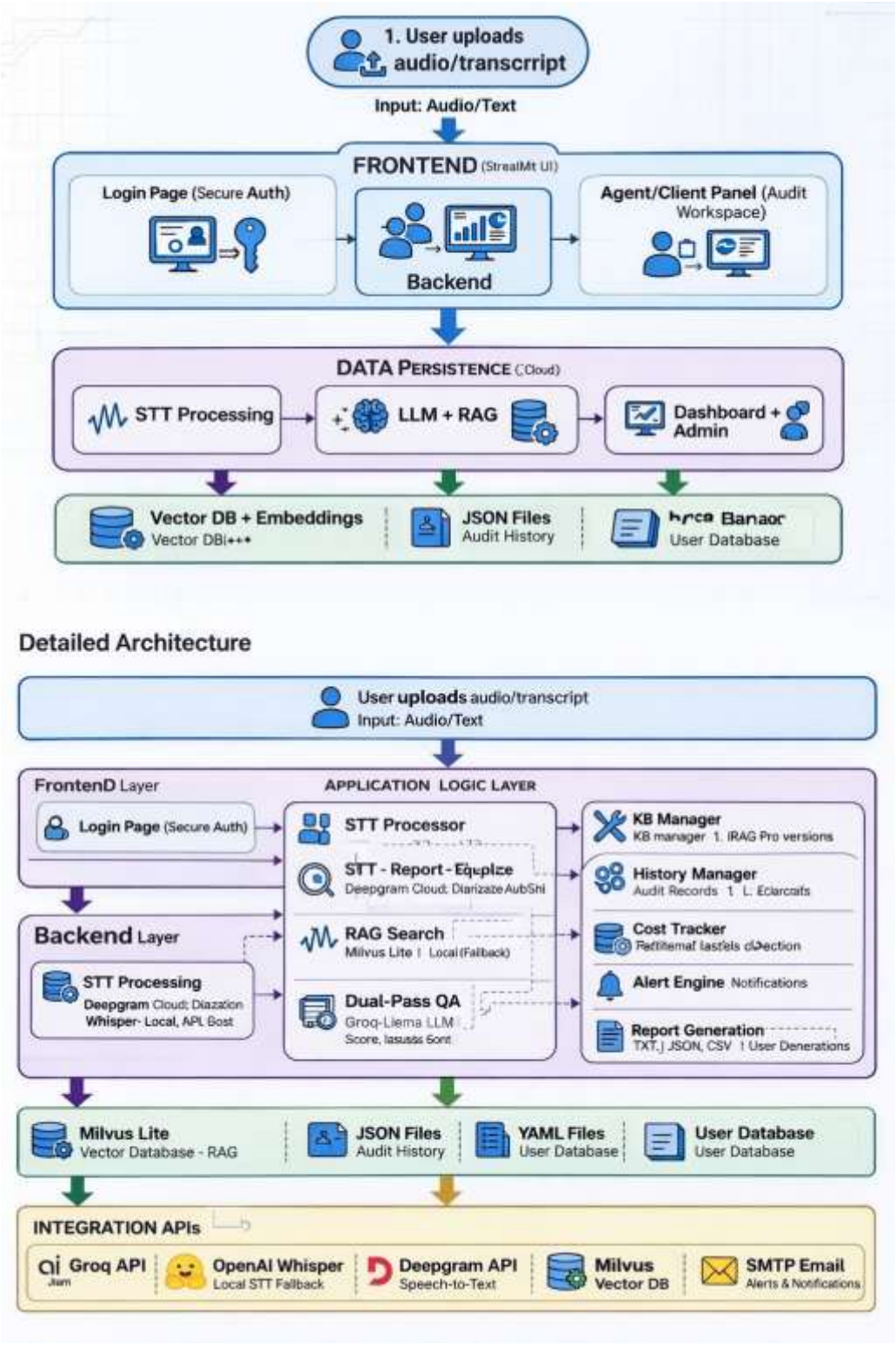
4.6 Generate User Password Hash

python generate_hash.py
Copy the generated bcrypt hash into:
.streamlit/secrets.toml

4.7 Run the Application

streamlit run app.py
Access the app at:
http://localhost:8501

5. System Architecture



5.1 Authentication Flow

The application presents four distinct screens based on authentication state:

1. **Landing page** — Marketing overview with hero headline, three feature cards (Auto transcription, Policy-aware scoring, Analytics dashboard), social proof stats (98.4% accuracy, <30s audit time, 14k+ daily calls), and a testimonial quote. Sign In and Get Started CTAs navigate to their respective auth screens.
2. **Login page** — Split-panel layout. Left panel: dark navy background with SamiX logo, tagline, testimonial quote, and three stat pills. Right panel: email + password fields, Forgot password link, Sign In button, and a link to Create account. Errors are shown inline only on failed submit — no pre-emptive banners.
3. **Signup page** — Same split-panel layout. Left panel: feature highlight checklist. Right panel: first name, last name, work email, password, confirm password with field-level validation before submission.
4. **Dashboard** — After successful authentication, the full sidebar navigation and page router are rendered.

5.2 New Audit (Upload Page)

The Upload page is the primary agent workspace. It presents two input modes:

- **Audio file mode** — Drag-and-drop zone accepting WAV, MP3, M4A, OGG, FLAC files up to 200 MB. After upload, a 5-step pipeline visualizer shows the processing steps (pydub → Deepgram → Groq x2 → Milvus → Store) with live status updates.
- **Live call mode** — Microphone recording with real-time RAG suggestions panel. Active Suggestions populate as the conversation progresses, pulling relevant policy context from the knowledge base.

Analysis options available before running the audit:

- STT engine selection: Auto (Deepgram → Whisper), Deepgram only, or Whisper local
- Use Knowledge Base (RAG): toggle on/off
- Speaker Diarization: toggle on/off
- Call type: General Support, Billing Inquiry, Technical Issue, Escalation, Complaint

5.3 Session Detail (Results)

After analysis completes, the session detail page renders the full audit result:

Section	Contents
Session header	Filename · Timestamp · Performance score · Sensitivity · Violations · Turns · Duration · Pass/Fail badge
Audio playback	Embedded audio player for the original recording
Call intelligence	Primary query · Customer expectation · Key topics discussed
Transcript tab	Turn-by-turn conversation with AGENT / CUSTOMER labels and Deepgram confidence score per turn
Dual scoring tab	QA Score (0–100) · Customer Sentiment (0–10 per turn) with Plotly chart
Where it went wrong	Exact turn number · Verbatim quote · Wrong fact · Correct fact · Specific correction
RAG groundedness	Which policy chunks were retrieved and how they influenced the score
Feedback tab	Manual reviewer notes and override capability
Download tab	Export as TXT · JSON · CSV · Email report

5.4 QA Dashboard

The QA Dashboard provides team-level analytics across all audits:

- **KPI row**: Total Audits, Avg QA Score (with month-over-month delta), Avg CSAT, Critical Failures count
- **Score trend chart**: QA score and CSAT over the last 30 audits — Plotly line chart with target-75 reference line
- **Pass/Fail rate**: Segmented bar showing High (>=75), Review (50–74), and Fail (<50) proportions
- **Radar chart**: Five-axis quality profile — Empathy, Compliance, Resolution, Communication, Policy Adherence
- **Dimension insights**: Per-dimension average with Strong / Needs Work / Critical status badge

5.5 Admin Panel

The Admin panel provides five tabs covering operational management:

- Analytics — QA score trend, score distribution, recent audit table with sortable columns
- Users — Active user count, invite/reset password/export actions
- Billing — Total tokens used, API cost, revenue, margin, invoice email, CSV export
- Knowledge Base — Upload PDF/TXT documents, index into Milvus, delete indexed documents
- System Health — Live service status (Groq, Deepgram, Whisper, LangChain, Milvus, pydub) with pre-flight check runner

6. Module Reference

6.1 src/pipeline/groq_client.py — LLM Inference

Wraps the Groq API for dual-pass quality scoring. Falls back to structured mock data if the API key is missing, ensuring the UI is always testable without a live key.

Method	Description
audit(transcript, kb_context, extra_context)	Main audit method. Pass 1: QA scoring. Pass 2: wrong-turn detection. Returns a unified result dict.
is_live (property)	Returns True if a valid Groq API key (starting with 'gsk_') is configured.
_build_prompt(transcript, kb_context)	Constructs the dual-pass prompt with policy context injected.
_parse_response(raw)	Parses the LLM JSON response into the standard result schema.

6.2 src/pipeline/stt_processor.py — Speech-to-Text

Manages both Deepgram cloud and Whisper local transcription with automatic fallback.

Method	Description
transcribe(file, engine, diarize)	Primary transcription entry point. Routes to Deepgram or Whisper based on engine param and key availability.
has_deepgram (property)	True if a valid Deepgram API key is configured.
_deepgram_transcribe(file, diarize)	Sends audio to Deepgram with diarize=true. Returns turn-by-turn transcript with speaker labels.
_whisper_transcribe(file)	Runs openai-whisper locally. No speaker separation — all text attributed to single speaker.
_parse_chat(text)	Parses pasted transcript text (format: 'Agent: ...' / 'Customer: ...') into the standard turns list.

6.3 src/utils/kb_manager.py — RAG Knowledge Base

Manages the Milvus Lite vector store and LangChain retrieval chain. Handles document upload, indexing, and similarity search.

Method	Description
search(query, top_k=5)	Embeds the query using all-MiniLM-L6-v2, searches the Milvus collection, returns top-k policy chunks as a concatenated string.
add_document(file)	Chunks the uploaded PDF/TXT, embeds each chunk, and inserts into the Milvus collection.
list_documents()	Returns list of unique source filenames indexed in the collection.
remove_document(filename)	Deletes all vectors associated with the given source filename.
is_vector_enabled (property)	True if Milvus Lite is available; False falls back to keyword search.

6.4 src/utils/history_manager.py — Audit Records

Persists audit results as JSON files in data/history/. Filename consistency is enforced: the uploaded filename is preserved through the entire pipeline.

Method	Description
save(result_dict)	Serializes the result to JSON and saves to data/history/{filename}_{timestamp}.json.
get_all()	Loads and returns all audit records sorted by date descending.
get_recent(limit=30)	Returns the most recent N audit records.
get_by_filename(name)	Returns all records matching a given original filename.

6.5 src/ui/components.py — UI Component Library

Centralised reusable widget library. All panels import from this module — raw HTML is never duplicated.

Component	Description
page_header(title, subtitle, icon)	Full-width page header with optional icon and subtitle. Animate-in CSS class.
kpi_row(metrics: list[dict])	Row of KPI cards with left-border accents, optional delta indicators, and icons.
score_card(label, score, max_score, color)	Single score card with progress bar. Auto-colors by threshold: green ≥75, amber ≥50, red <50.
transcript_container(turns)	Scrollable chat-bubble transcript viewer. Agent bubbles left-aligned, Customer right-aligned with blue fill.
alert_banner(message, level)	Inline alert banner. Levels: info, success, warning, error.

Component	Description
empty_state(icon, title, subtitle)	Centered empty state with large icon for zero-data screens.
info_card(title, content, icon, accent)	Content card with optional left-border accent color.
cost_card(tokens, cost_usd, margin_usd)	API cost summary card with optional margin row.
sidebar_user_card(name, email, role)	User card with avatar initials badge and role pill.
system_status_panel(statuses)	Service status pills (Live/Mock/Local) for the sidebar.
labeled_divider(label)	Horizontal divider with centered text label.
render_gauge / render_three_gauges	Legacy aliases — map to score_card for backward compatibility.

7. API Keys and Configuration

7.1 Required and Optional Keys

Service	Required / Optional	Free Tier & Notes
Groq	Required	~14,400 req/day free — console.groq.com
Deepgram	Optional	$200 credit free — console.deepgram.com
Whisper	Built-in	Free local model — pip install openai-whisper
Milvus Lite	Built-in	Free local DB — pip install pymilvus[milvus_lite]

7.2 Environment Variables (.env)

```
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxx
DEEPGRAM_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
SMTP_HOST=smtp.gmail.com          # optional — for email alerts
SMTP_PORT=587                     # optional
SMTP_USER=your@email.com          # optional
SMTP_PASS=your_app_password       # optional
```

7.3 Streamlit Secrets (.streamlit/secrets.toml)

Used for cloud deployment (Streamlit Cloud, Heroku). Never commit this file.

```
[api_keys]
groq_api_key = "gsk_xxxxxxxxxxxx"
deepgram_api_key = "xxxxxxxx"

[auth]
admin_email = "admin@samix.ai"
admin_hash  = "$2b$12$..."  # generated by generate_hash.py
```

8. Deployment

8.1 Local Development

```
streamlit run app.py
```

Opens at http://localhost:8501. Hot-reload is enabled — save any .py file to see changes immediately.

8.2 Streamlit Cloud (Recommended for demos)

5. Push your repository to GitHub (secrets.toml and .env are excluded by .gitignore)
6. Go to share.streamlit.io and click New app
7. Select your repository, branch (main), and entry file (app.py)
8. Click Advanced settings and paste your secrets.toml content
9. Click Deploy — live in approximately 2 minutes

8.3 Docker

```
docker-compose up --build
```

Builds the container, installs all dependencies, and starts the app on port 8501. The docker-compose.yml mounts the data/ directory as a volume so audit history persists across restarts.

8.4 Environment Matrix

Platform	Command	Notes
Local	streamlit run app.py	Development — hot reload enabled
Streamlit Cloud	Automatic from GitHub push	Free tier — recommended for public demos
Docker	docker-compose up --build	Production — data volume persisted
Heroku	git push heroku main	PaaS — use config vars for secrets
AWS / GCP	See DEPLOYMENT.md	Container or VM — full control

9. Troubleshooting

Error / Symptom	Cause	Fix
pydub AudioSegment error	FFmpeg not in system PATH	Download FFmpeg from ffmpeg.org and add the bin/ folder to your system PATH environment variable
cannot import name 'render_gauge'	Old panel file importing from old component names	Replace src/ui/components.py with the latest version which includes legacy alias functions
Milvus ConnectionConfigException	pymilvus[milvus_lite] not installed	Run: pip install "pymilvus[milvus_lite]" — app falls back to keyword search automatically
Groq mock data showing	API key missing or invalid	Check .env or secrets.toml — key must start with 'gsk_' (not 'gsk_REPLACE')
Login not working	bcrypt hash mismatch	Re-run python generate_hash.py and copy the new hash exactly into secrets.toml
streamlit-authenticator import error	Wrong package version	Run: pip install streamlit-authenticator==0.3.2
sentence-transformers slow first run	Downloading MiniLM model on first use	Normal behaviour — model (~90 MB) downloads once to ~/.cache/huggingface
Deepgram MOCK showing in sidebar	Deepgram API key not configured	Add DEEPGRAM_API_KEY to .env — app continues working with Whisper fallback

10. Data Schemas

10.1 Audit Result JSON Schema

Every audit produces a JSON object with the following fields, saved to data/history/:

Field	Type	Description
filename	string	Original uploaded filename — preserved throughout pipeline
date	string	ISO 8601 timestamp of the audit
agent_name	string	Agent name extracted from transcript or user profile
qa_score	float 0–100	Overall agent quality score from Groq pass 1
empathy_score	float 0–100	Empathy sub-score
compliance_score	float 0–100	Policy compliance sub-score
resolution_score	float 0–100	Issue resolution sub-score
csat_score	float 0–10	Estimated customer satisfaction per turn
summary	string	One-paragraph call summary
issues	list[string]	Identified problems in agent behaviour
recommendations	list[string]	Specific improvement suggestions
turns	list[dict]	Turn-by-turn transcript: speaker, text, sentiment, confidence
wrong_turns	list[dict]	turn, quote, wrong, correct — from pass 2
kb_chunks_used	list[string]	Policy document chunks retrieved by RAG
tokens_used	int	Total tokens consumed by both Groq calls
cost_usd	float	Estimated API cost in USD
margin_usd	float	Revenue minus cost for this audit
has_alert	bool	True if QA score < 60 and an alert was triggered
duration	string	Call duration MM:SS

10.2 User Record Schema (data/auth/users.yaml)

```yaml
users:
  - name: Evu Dayasree
    email: evu@samix.ai
    password: $2b$12$...  # bcrypt hash — generated by generate_hash.py
    role: agent           # agent | admin
```

11. Security Considerations

Concern	Implementation
Password storage	bcrypt hashing via streamlit-authenticator — plaintext passwords never stored
API key storage	.env and secrets.toml excluded from git via .gitignore — never committed
Session management	Streamlit session_state — server-side, cleared on logout
Audio file storage	Uploaded files saved to data/uploads/ — not exposed publicly
Audit history	Stored as local JSON files — not transmitted externally
SMTP credentials	Stored in .env only — never hardcoded or logged
RAG documents	Stored locally in Milvus Lite — not sent to external services

Production checklist

Use strong, unique passwords for all user accounts.
Rotate Groq and Deepgram API keys regularly.
Enable HTTPS when deploying behind a reverse proxy (nginx/Caddy).
Restrict access to the data/ directory — set filesystem permissions to 700.
Enable SMTP TLS (port 587 with STARTTLS) for email alerts.
Review .gitignore before every push — confirm .env and secrets.toml are excluded.

12. Roadmap and Future Enhancements